

VIT BHOPAL UNIVERSITY

CampusTracker

Student Academic Management System

PROJECT REPORT

Submitted By

Chirag Patil

25BAI10151

Course

Programming in Java

Academic Year 2026

1 Introduction

CampusTracker is a console-based Student Academic Management System developed using Java. The system is designed to manage student records, courses, enrollments, attendance, and academic reports through a simple command-line interface.

The project applies important Java programming concepts such as object-oriented programming, collections, file handling, exception handling, input validation, and multithreading.

The application follows a menu-driven workflow that allows users to perform different academic management operations from a single system.

2 Problem Statement

Managing academic records manually can become difficult when the number of students, courses, enrollments, and attendance records increases.

A suitable academic management system should provide a structured way to:

- Store student information.
- Store course information.
- Manage student enrollment.
- Record attendance.
- Calculate attendance percentage.
- Generate academic reports.
- Save information for future use.
- Handle invalid input and errors.

CampusTracker provides a console-based solution for these activities.

3 Objectives

The main objectives of the project are:

1. To create a student academic management system using Java.
2. To manage student and course records.
3. To manage student enrollments.
4. To record and calculate attendance.

5. To generate academic reports.
6. To store application data using file handling.
7. To provide input validation and exception handling.
8. To demonstrate the use of multithreading.

4 Scope of the Project

The scope of CampusTracker includes:

- Student management.
- Course management.
- Enrollment management.
- Attendance management.
- Academic reporting.
- Dashboard information.
- File-based data persistence.
- Automatic data saving.

The project can later be extended into a larger academic management application with database support, authentication, graphical interfaces, and online access.

5 Target Users

The intended users of the system are:

- Students.
- Faculty members.
- Academic staff.
- System administrators.

6 System Requirements

6.1 Functional Requirements

6.1.1 Student Management

The student management module allows users to:

- Add student records.
- View student records.
- Update student information.
- Delete student records.
- Validate student information.

6.1.2 Course Management

The course management module provides:

- Adding courses.
- Viewing courses.
- Updating course information.
- Deleting courses.
- Maintaining course code, title, credits, and faculty information.

6.1.3 Enrollment Management

The enrollment module allows users to:

- Enroll students into courses.
- View enrollment records.
- Validate student and course identifiers.

6.1.4 Attendance Management

The attendance module stores the number of classes attended and total classes.

Attendance percentage is calculated using:

$$Attendance\ Percentage = \frac{Classes\ Attended}{Total\ Classes} \times 100$$

The system also provides an attendance status based on the configured eligibility condition.

6.1.5 Reporting

The reporting module provides:

- Student reports.
- Course reports.
- Enrollment information.
- Attendance summaries.
- Dashboard information.

6.1.6 Data Persistence

The system stores application information in files so that records can be loaded again when the application starts.

6.1.7 Automatic Save

The system uses a background thread to periodically save application data.

6.2 Non-Functional Requirements

6.2.1 Usability

The menu-driven console interface provides a simple way for users to select and perform operations.

6.2.2 Reliability

Input validation and exception handling are used to handle expected errors and maintain application continuity.

6.2.3 Performance

Java collections are used for efficient storage and processing of records during execution.

6.2.4 Maintainability

Responsibilities are divided among multiple classes and service components, making the system easier to modify.

6.2.5 Error Handling

The application handles invalid values, missing records, invalid operations, and other expected input errors.

6.2.6 Resource Efficiency

The file-based approach keeps the project lightweight and avoids the requirement of a separate database server.

7 System Architecture

CampusTracker follows a simple layered logical architecture.

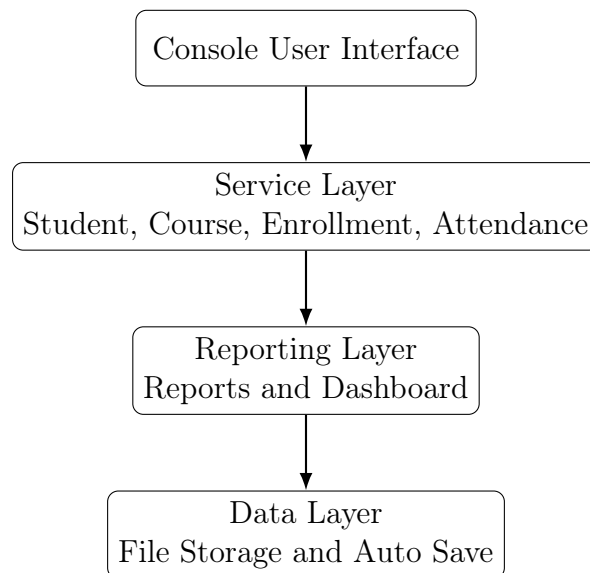


Figure 1: System Architecture of CampusTracker

8 System Workflow

The general workflow of the application is:

1. Start the application.
2. Load stored data.
3. Display the main menu.
4. Read the user's choice.
5. Validate the input.
6. Perform the selected operation.

7. Display the result.
8. Save updated data.
9. Continue until the user selects Exit.

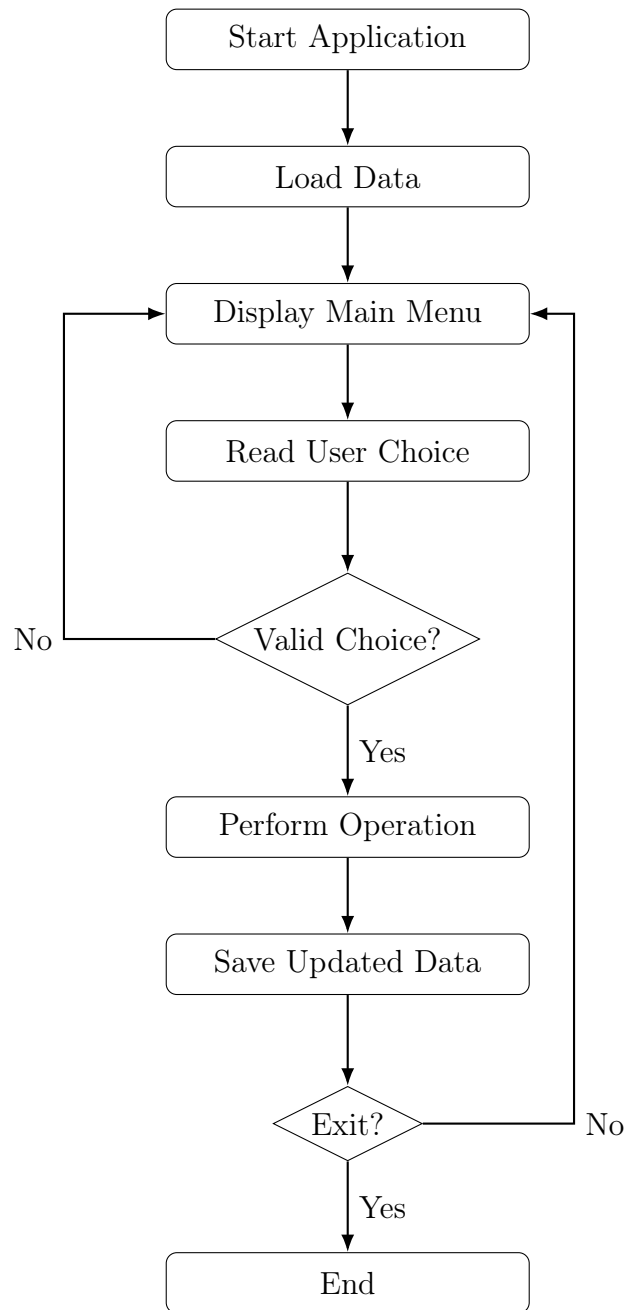


Figure 2: CampusTracker Workflow

9 Use Case Design

The main actor in the system is the user.

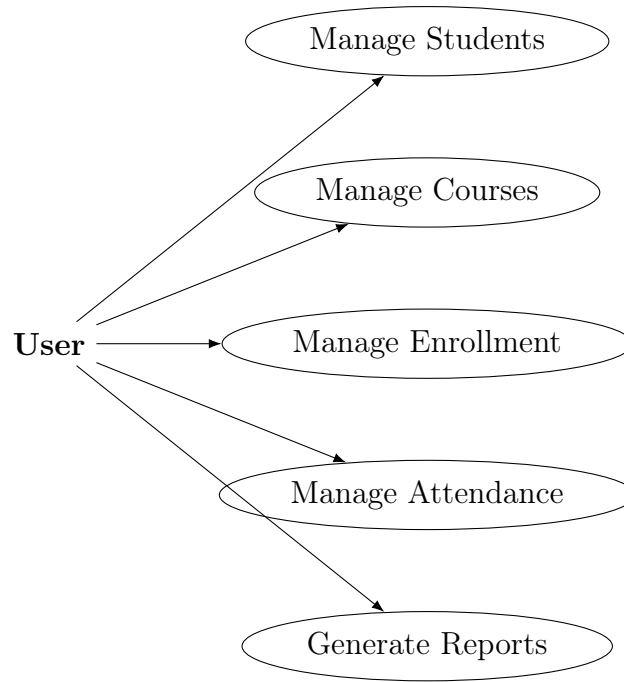


Figure 3: Use Case Diagram

10 Class and Component Structure

The major classes and components used in the project are:

- **St** – Student entity.
- **Cr** – Course entity.
- **En** – Enrollment entity.
- **At** – Attendance entity.
- **Ss** – Student service.
- **Cs** – Course service.
- **Es** – Enrollment service.
- **As** – Attendance service.
- **Rs** – Reporting service.
- **Ds** – Data storage.
- **Ast** – Automatic save thread.

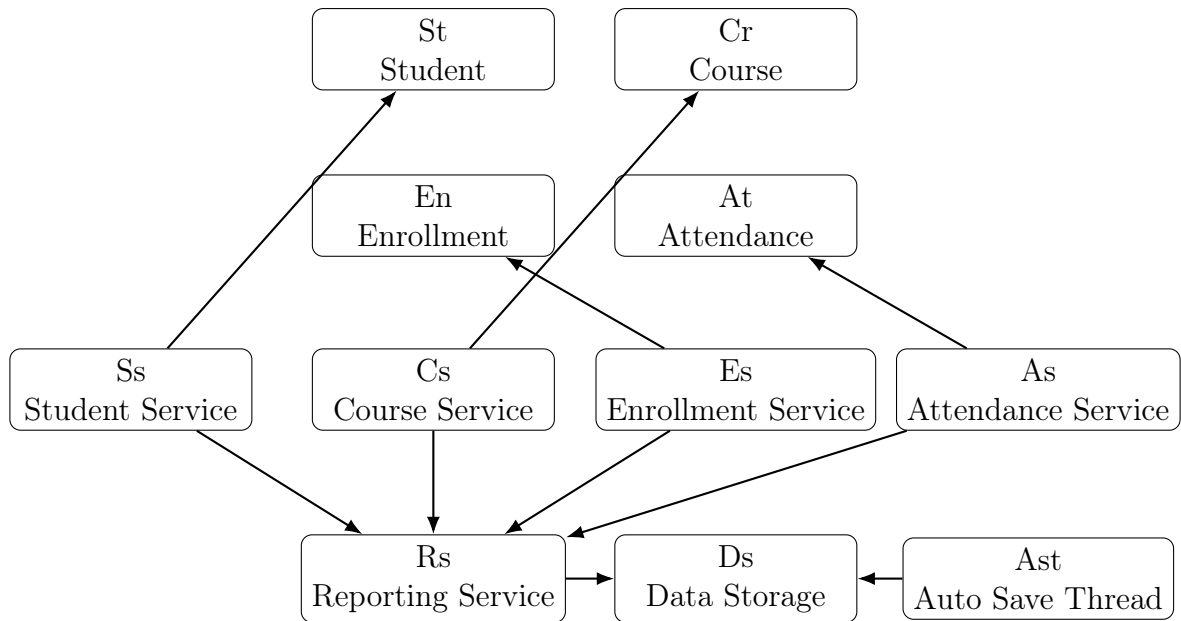


Figure 4: Class and Component Structure

11 Sequence Design

The sequence below represents a typical user operation.

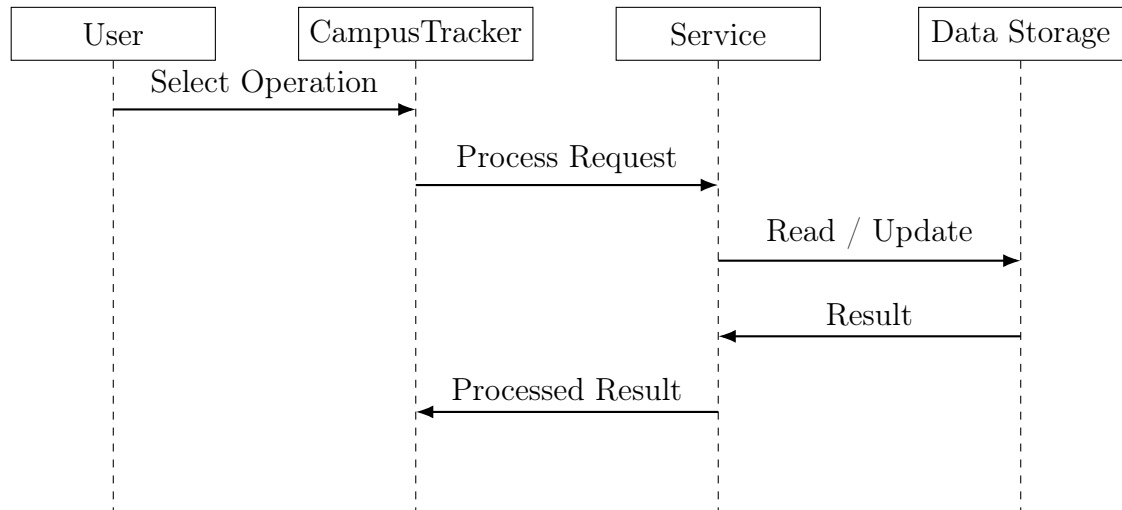


Figure 5: Sequence Diagram

12 Data Storage Design

CampusTracker uses file-based storage for maintaining application information.

Data Type	Stored Information
Student	Student ID, name, email, department, semester
Course	Course code, title, credits, faculty
Enrollment	Student ID and course code
Attendance	Student ID, course code, attended classes, total classes

Table 1: Data Storage Structure

The `Ds` component is responsible for loading and saving the stored records.

13 Technology Used

Technology / Concept	Purpose
Java	Main programming language
Maven	Project build management
Java Collections	Record storage and processing
File I/O	Data persistence
Exception Handling	Error handling
Multithreading	Automatic save functionality
OOP	Class-based system design
Git and GitHub	Version control and project hosting

Table 2: Technologies Used

14 Implementation

14.1 Package Structure

The application uses the package:

```
package com.mycompany.campustracker;
```

14.2 Main Class

The main class of the project is:

```
public class CampusTracker
```

The application starts using:

```
public static void main(String[] args){new CampusTracker().start();}
```

14.3 Student Entity

The student class stores:

- Student ID.
- Student name.
- Email.
- Department.
- Semester.

14.4 Course Entity

The course class stores:

- Course code.
- Course title.
- Credits.
- Faculty.

14.5 Enrollment Entity

The enrollment class connects a student with a course using their identifiers.

14.6 Attendance Entity

The attendance class stores student ID, course code, attended classes, and total classes.

14.7 Service Classes

Class	Responsibility
Ss	Student operations
Cs	Course operations
Es	Enrollment operations
As	Attendance operations
Rs	Reporting operations
Ds	Data persistence
Ast	Automatic save thread

Table 3: Service Classes

15 Object-Oriented Concepts

15.1 Classes and Objects

The project uses classes to represent students, courses, enrollments, attendance, and services.

15.2 Encapsulation

Data and the operations performed on that data are grouped within suitable classes.

15.3 Abstraction

Service components hide implementation details from the main application workflow.

15.4 Modularity

Different responsibilities are separated among different classes and components.

16 Collections Used

Java collections are used for storing and processing application records.

Examples include:

- `ArrayList` for record storage.
- `HashMap` for key-based lookup where required.

17 Exception Handling

Exception handling is used to manage expected runtime and input-related errors.

The general processing flow is:

$$Input \rightarrow Validation \rightarrow Operation \rightarrow Output$$

For invalid input:

$$Input \rightarrow Error\ Handling \rightarrow Recovery \rightarrow Menu$$

This allows the program to continue running after expected errors.

18 Multithreading

The `Ast` class is used for automatic saving.

The thread periodically performs save operations while the main application continues to accept user input.

The basic process is:

Start Thread -> Wait -> Save Data -> Repeat

19 Application Screenshots



Figure 6: CampusTracker Main Menu

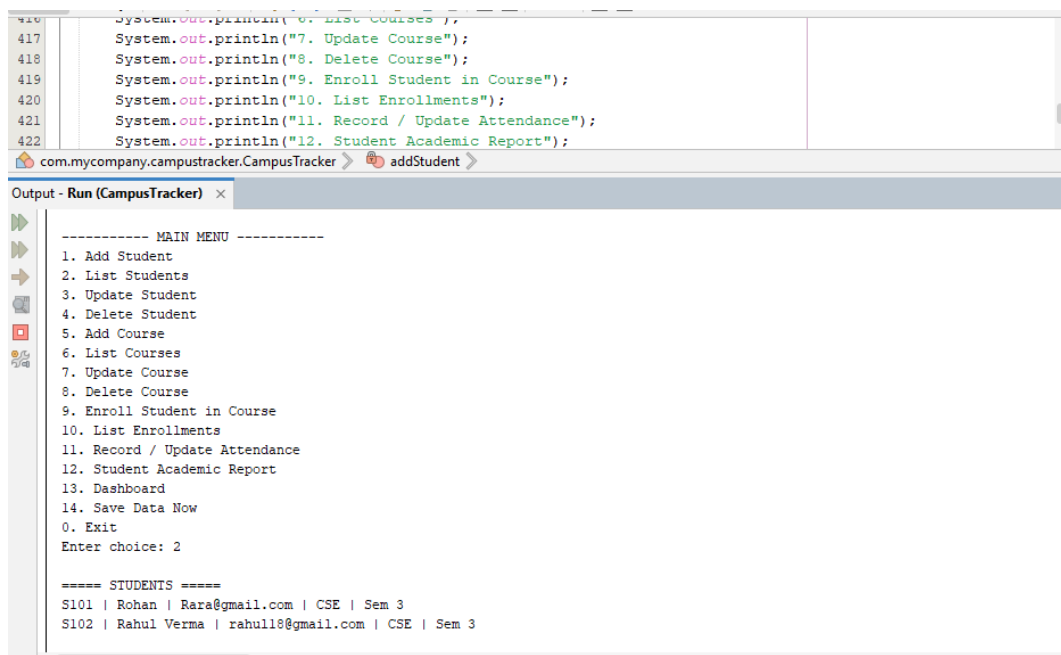


Figure 7: Student Management

```
422 | System.out.println("12. Student Academic Report");
com.mycompany.campus tracker.CampusTracker > addStudent >
Output - Run (CampusTracker) x
----- MAIN MENU -----
1. Add Student
2. List Students
3. Update Student
4. Delete Student
5. Add Course
6. List Courses
7. Update Course
8. Delete Course
9. Enroll Student in Course
10. List Enrollments
11. Record / Update Attendance
12. Student Academic Report
13. Dashboard
14. Save Data Now
0. Exit
Enter choice: 11
Student ID: S101
Course code: Java2006
Classes attended: 18
Total classes: 20
Attendance saved.
```

Figure 8: Attendance Management

```
1. Add Student
2. List Students
3. Update Student
4. Delete Student
5. Add Course
6. List Courses
7. Update Course
8. Delete Course
9. Enroll Student in Course
10. List Enrollments
11. Record / Update Attendance
12. Student Academic Report
13. Dashboard
14. Save Data Now
0. Exit
Enter choice: 12
Student ID: S101
===== STUDENT REPORT =====
Student: Rohan (S101)
Department: CSE | Semester: 3

Course Enrollments:
- Java2006 : OOPS | Attendance: 90.00%
```

Figure 9: Academic Report

```
3. Update Student
4. Delete Student
5. Add Course
6. List Courses
7. Update Course
8. Delete Course
9. Enroll Student in Course
10. List Enrollments
11. Record / Update Attendance
12. Student Academic Report
13. Dashboard
14. Save Data Now
0. Exit
Enter choice: 13
===== ACADEMIC DASHBOARD =====
Students      : 2
Courses       : 1
Enrollments   : 1
Attendance rows: 1
Eligible rows : 1
Departments   : {CSE=2}
```

Figure 10: Dashboard

20 Testing

Testing was carried out using functional test cases covering the major operations.

ID	Test Case	Expected Result	Status
T1	Add valid student	Student added successfully	Pass
T2	Add valid course	Course added successfully	Pass
T3	Enroll valid student	Enrollment created	Pass
T4	Invalid student ID	Validation message displayed	Pass
T5	Invalid course code	Validation message displayed	Pass
T6	Record attendance	Attendance updated	Pass
T7	Generate report	Report displayed	Pass
T8	Save and reload data	Records recovered	Pass
T9	Invalid menu input	Error handled	Pass
T10	Automatic save	Data saved periodically	Pass

Table 4: Functional Test Cases

21 Results

The implemented system provides a working console-based academic management workflow.

The major results are:

- Student records can be managed.
- Course records can be managed.
- Students can be enrolled into courses.
- Attendance can be recorded and calculated.
- Reports can be generated.
- Data can be stored and loaded using files.
- Automatic saving is supported using a background thread.
- Invalid operations are handled using validation and exception handling.

22 GitHub Repository

The project is hosted on GitHub at:

<https://github.com/Chirag-Patil11/CampusTracker>

The repository contains the source code, documentation, configuration files, and supporting project files.

Git is used to maintain versions of the project during development.

23 Project Structure

The project follows a Maven-based structure:

```
CampusTracker/  
|  
|-- src/  
| |-- main/  
| |-- java/  
| |-- com/  
| |-- mycompany/  
| |-- campustracker/  
| |-- CampusTracker.java  
|  
|-- docs/  
|-- data/  
|-- pom.xml  
|-- README.md  
|-- statement.md  
|-- .gitignore
```

24 Build and Execution

The project can be built using Maven.

```
mvn clean package
```

The application can be executed using:

```
java -cp target/classes com.mycompany.campustracker.CampusTracker
```

The project can also be opened and executed through NetBeans.

25 Design Decisions

25.1 Console Interface

A console-based interface was selected to provide a simple and direct interaction model.

25.2 File-Based Storage

File storage keeps the application lightweight and avoids dependency on a separate database server.

25.3 Service-Based Organization

Different responsibilities such as students, courses, enrollment, attendance, reports, and storage are handled through separate components.

25.4 Automatic Save

A background thread was included to periodically save application data.

26 Challenges Faced

26.1 Input Validation

Handling incorrect input while keeping the program active required suitable validation and exception handling.

26.2 Data Persistence

The application needed a reliable mechanism for saving and loading academic records.

26.3 Class Organization

The project contains multiple entities and services, which required proper organization of responsibilities.

26.4 Multithreading

Implementing automatic saving required understanding how the background thread operates alongside the main application.

26.5 GitHub Management

GitHub setup required maintaining the correct repository structure and controlling generated files.

27 Learning Outcomes

This project provided practical experience in:

- Java programming.
- Object-oriented programming.
- Classes and objects.
- Collections.
- File input and output.
- Exception handling.
- Input validation.
- Multithreading.
- Modular programming.
- Maven.
- Git and GitHub.
- Software testing.

28 Future Enhancements

Possible future improvements include:

- Adding a graphical user interface.
- Replacing file storage with a database.
- Adding authentication and authorization.
- Adding role-based access.
- Adding marks and grade management.
- Adding advanced academic analytics.
- Adding report export functionality.
- Developing a web-based version.

29 Conclusion

CampusTracker demonstrates the development of a Student Academic Management System using Java.

The project combines object-oriented programming, collections, file handling, exception handling, input validation, reporting, and multithreading in a single application.

The system provides a structured way to manage student academic information through a console interface and demonstrates practical software development concepts including modular design, testing, documentation, and GitHub-based version control.

30 References

1. VIT Bhopal University, Programming in Java Project Guidelines.
2. Oracle Java Documentation.
3. Apache Maven Documentation.
4. Git Documentation.
5. GitHub Documentation.

A Project Features

Feature	Description
Student Management	Add, view, update, and delete student records.
Course Management	Add, view, update, and delete course records.
Enrollment	Connect students with courses.
Attendance	Record attendance and calculate percentage.
Reports	Generate academic reports.
Dashboard	Display academic information.
Persistence	Store information in files.
Auto Save	Periodically save data using a thread.
Validation	Check user input before processing.
Exception Handling	Handle expected input and runtime errors.